CS603 – Software Architecture & Design

Final Term Papers 2025

Prepared By: Usama Farooq

Short Questions

1. Write three properties of system at run time.
2. Enlist benefits of refactoring.
3. Write two points to improve availability of system in cloud computing.
4. What are two different deployment methods available on cloud computing?
5. How can we make refactoring safe?
6. Write three bad smells of change preventers category.
7. If a diagram is given, how to tell its view type and system type?
8. Two types of component and connector type patterns.
9. Write any two benefits of designing a consistent navigation structure of web application.
10. Write benefits of good design.
11. Identify the key factors of service-oriented architecture.
12. Write types of allocations.
13. Choose the answer for following questions. (State, Behavior, Identity)
    i. Current values of properties of class
    ii. Part of system intelligence delegated to the object
    iii. Property of object which distinguish it from all other object
14. Difference between refactoring and performance optimization.
15. Write the characteristics of cloud computing.
16. Note on object-orientation services.
17. Is it remove Temp with Query? Give reason with yes or no.
18. How to replace Temp with Query (give example)
19. If a person built a PVT. LTD company, then what policies of refactoring structure should apply?
20. Write three names of change preventer.
21. Define the concept sidecar with the help of a diagram.
22. Which Structure is strongly tied to the achievement of performance, availability, and security goals?
23. Which of the following are the possible refactorings applied on duplicated code?
    i. Extract method
    ii. Move method
    iii. Extract class
    iv. Abstract class
    v. Pull Up method
    vi. Inherited class
24. Martin Fowler has divided the known twenty-two bad smells into five categories and “Bloaters” is one of them. Write down the names of any three bad smells that fall in the Bloaters category.
25. You should design a data layer for your web application to abstract the logic which is necessary to access the database. What are the benefits of using a separate data layer?
26. Patterns can be characterized according to the type of architectural elements they use. You are required to enlist any two allocation type patterns.

Long Questions

1. How to find refactor by extract method? Write steps.
2. Write deployment models of cloud computing.
3. Brief note on problems of refactoring.
4. If a scenario is given, how to know architecture pattern? Explain different architecture patterns.
5. Describe scenario module system. Also tell development and maintenance of independently system modules.
6. If you're the owner, which group you choose?
   Group A: Fast delivery without documentation

Usama Farooq

Group B: Complete documentation. Describe with reason.

7. Bad smells are categorized in five different types, describe them.
8. What factors influence your choice of architecture style?
9. Write key benefits of service-oriented architecture.
10. If a object b2 of class b id created the object a5 of class a is also created automatically. What is the lifetime of the a5 object?
11. If there is a abstract class or api, the implementer are not required to implement the functionality which they are not interested in what type of principle implement in this scenario?
12. Three principles lead to the dependency inversion principle write the name of them.
13. Suppose there is only one line of code. Separate the true or false statement from the given
    a. Bug is hard to find.
    b. Hard for the new developer.
    c. It is easy to understand.
    d. It is easy to fix.
    e. It is easy to amendable for the future implementation.
14. Note on allocation structure useful factors.
15. Explain internal desirable design characteristics of software design process.
16. Write types of dispensable and shortly explain each.
17. Which types of guidelines should be followed at the time of designing request processing strategy for a web application? Explain.
18. Enlist at least four uses of Software Architecture Patterns?
19. Dispensable is a category of bad smell in code, identify the different types of dispensables in code from the given options.
    I. Comments
    II. Move method
    III. Extract method
    IV. Duplicate code
    V. Lazy class
    VI. Abstract class
20. 4+1 View Model of Architecture is given below which represents the four different views, you are required to identify the viewers of each type of view and mention their names.

Answers:

Short Questions

1. **Three properties of a system at runtime**
   Performance, availability, scalability.
2. **Benefits of refactoring**
   Improves code readability, reduces complexity, enhances maintainability, increases code reusability.
3. **Two points to improve availability in cloud computing**
   Use redundant systems across multiple availability zones; implement automated failover mechanisms.
4. **Two different deployment methods in cloud computing**
   Blue-green deployment, canary deployment.
5. **How to make refactoring safe**
   Use automated tests to verify functionality; apply small, incremental changes with version control.
6. **Three bad smells of change preventers category**
   Divergent change, shotgun surgery, parallel inheritance hierarchies.
7. **Identify view type and system type from a diagram**
   Check diagram elements (e.g., components, modules, processes) and context (e.g., runtime, structural); view types include module, component-and-connector, allocation; system types include distributed, monolithic, etc.
8. **Two types of component and connector patterns**
   Publish-subscribe, client-server.
9. **Two benefits of consistent navigation structure in web applications**
   Enhances user experience, reduces learning curve for users.

Usama Farooq

10. **Benefits of good design**
    Improves maintainability, scalability, and reduces technical debt.
11. **Key factors of service-oriented architecture (SOA)**
    Loose coupling, service reusability, interoperability, discoverability.
12. **Types of allocations**
    Deployment allocation, work allocation, implementation allocation.
13. **Choose the answer (State, Behavior, Identity)**
    i. Current values of properties of class: **State**
    ii. Part of system intelligence delegated to the object: **Behavior**
    iii. Property of object distinguishing it from others: **Identity**
14. **Difference between refactoring and performance optimization**
    Refactoring improves code structure without changing functionality; performance optimization enhances speed or resource usage, often altering behavior.
15. **Characteristics of cloud computing**
    On-demand self-service, scalability, resource pooling, measured service, broad network access.
16. **Note on object-orientation services**
    Encapsulation, inheritance, polymorphism, and abstraction provide modularity and reusability in software design.
17. **Is it "Remove Temp with Query"? Reason**
    Yes, it eliminates temporary variables by replacing them with method calls, improving readability and reducing errors.
18. **How to replace Temp with Query (example)**
    Before: double basePrice = order.getQuantity() * order.getPrice(); if (basePrice > 1000) return basePrice * 0.95;
    After: if (order.getBasePrice() > 1000) return order.getBasePrice() * 0.95; where getBasePrice() is a method.
19. **Refactoring structure policies for a Pvt. Ltd. company**
    Use incremental refactoring, maintain test coverage, prioritize high-impact areas, and document changes.
20. **Three names of change preventers**
    Divergent change, shotgun surgery, parallel inheritance hierarchies.
21. **Concept of sidecar with diagram**
    The sidecar pattern deploys helper components alongside a main application to handle cross-cutting concerns (e.g., logging, monitoring).
    [Main App] <--> [Sidecar: Logging/Monitoring]
22. **Structure tied to performance, availability, and security goals**
    Component-and-connector structure.
23. **Possible refactorings for duplicated code**
    i. Extract method
    iii. Extract class
    v. Pull Up method
24. **Three bad smells in Bloaters category**
    Long method, large class, primitive obsession.
25. **Benefits of a separate data layer**
    Abstracts database logic, improves maintainability, enhances scalability, and isolates data access concerns.
26. **Two allocation type patterns**
    Deployment pattern, work assignment pattern.

Long Questions

1. **How to find refactor by extract method? Write steps.**
   To identify refactoring opportunities using the extract method technique, start by reviewing the code for large or repetitive methods. Break down the logic into smaller, self-contained units by identifying a distinct piece of functionality (e.g., a calculation or loop). Create a new method with a descriptive name, move the extracted code into it, and replace the original code with a call to this method. Finally, run automated tests to ensure the refactoring preserves the original behavior and functionality.
2. **Write deployment models of cloud computing.**
   Cloud computing offers several deployment models: **Public cloud** provides shared resources over the internet (e.g., AWS, Azure), accessible to anyone. **Private cloud** offers dedicated resources for a single organization, ensuring higher security and control. **Hybrid cloud** combines public and private clouds for flexibility, allowing

Usama Farooq

data and applications to move between them. **Community cloud** is shared among specific organizations with common needs, optimizing costs and resources.

3. **Brief note on problems of refactoring.**
   Refactoring can introduce unintended bugs if not accompanied by comprehensive testing, as changes might affect dependent code. It requires significant time and effort, potentially delaying project timelines, and may face resistance from teams unfamiliar with its benefits. Additionally, poor documentation during refactoring can lead to confusion, and over-refactoring might result in unnecessary complexity or technical debt if not aligned with project goals.
4. **If a scenario is given, how to know architecture pattern? Explain different architecture patterns.**
   To identify an architecture pattern, analyze the scenario's requirements (e.g., scalability, modularity, real-time processing). **Layered architecture** separates concerns like presentation, business logic, and data access. **Client-Server** involves a central server handling requests from clients. **Microservices** break the system into independent, deployable services. **Event-Driven** uses events to trigger actions, suitable for asynchronous systems. **MVC** (Model-View-Controller) separates data (Model), user interface (View), and control logic (Controller).
5. **Describe scenario module system. Also tell development and maintenance of independently system modules.**
   A scenario module system organizes software into independent modules, each handling specific functionality (e.g., user authentication, payment processing). Development involves defining clear interfaces, minimizing dependencies, and using modular design principles like encapsulation. Maintenance requires independent updates to modules, version control for changes, and ensuring compatibility through well-defined APIs, allowing parallel development and easier bug fixes.
6. **If you're the owner, which group you choose? Group A: Fast delivery without documentation Group B: Complete documentation. Describe with reason.**
   As an owner, I would choose **Group B: Complete documentation**. While **Group A** offers fast delivery, the lack of documentation increases long-term risks, such as difficulty in maintenance, onboarding new developers, and scaling the system. Documentation ensures knowledge transfer, reduces technical debt, and supports sustainability, outweighing the initial speed advantage of Group A.
7. **Bad smells are categorized in five different types, describe them.**
   Bloaters include large methods or classes that are hard to maintain (e.g., long method). **Object-Oriented Abusers** involve misuse of OO principles, like excessive switch statements. **Change Preventers** resist modifications (e.g., divergent change). **Dispensables** are unnecessary code elements (e.g., dead code). **Couplers** indicate tight coupling issues (e.g., feature envy), complicating changes.
8. **What factors influence your choice of architecture style?**
   The choice of architecture style depends on system requirements (e.g., scalability, performance), the team's expertise in implementing the style, budget constraints, the deployment environment (e.g., cloud, on-premises), and non-functional needs like security or maintainability. User expectations and future growth potential also play a critical role in this decision.
9. **Write key benefits of service-oriented architecture.**
   Service-Oriented Architecture (SOA) offers loose coupling, allowing services to operate independently, enhancing flexibility. It promotes service reusability across projects, reducing development time. SOA ensures interoperability across different platforms and technologies, and supports incremental development, improving scalability and adaptability to business needs.
10. **If an object b2 of class B creates the object a5 of class A automatically, what is the lifetime of the a5 object?**
    The lifetime of object a5 depends on the scope of object b2. If a5 is created as an instance variable within b2, it persists as long as b2 exists (e.g., until b2 is garbage-collected or destroyed). If a5 is a local variable, its lifetime ends when the method creating it in b2 completes, unless explicitly managed otherwise.
11. **If there is an abstract class or API, the implementers are not required to implement the functionality they are not interested in, what type of principle implements in this scenario?**
    This scenario implements the **Interface Segregation Principle (ISP)**, which states that clients should not be forced to depend on interfaces they do not use. By allowing implementers to ignore irrelevant methods in an abstract class or API, ISP promotes cleaner, more focused code and reduces unnecessary complexity.
12. **Three principles lead to the Dependency Inversion Principle, write the names of them.**
    The **Dependency Inversion Principle (DIP)** is derived from three principles: **Open-Closed Principle** (extend without modifying), **Liskov Substitution Principle** (subtypes must be substitutable for their base types), and **Single Responsibility Principle** (a class should have one reason to change), collectively supporting flexible, decoupled designs.

Usama Farooq

13. **Suppose there is only one line of code. Separate the true or false statement from the given:**
    a. Bug is hard to find: **False** (single line is easier to debug).
    b. Hard for the new developer: **False** (simple and clear for beginners).
    c. It is easy to understand: **True** (minimal complexity aids comprehension).
    d. It is easy to fix: **True** (isolated issue is simpler to resolve).
    e. It is easy to amend for future implementation: **True** (small code base adapts easily).
14. **Note on allocation structure useful factors.**
    Allocation structures map software components to hardware (deployment), teams (work allocation), or development processes (implementation). They optimize resource utilization, enhance performance by aligning with hardware capabilities, and improve scalability and maintenance by clarifying responsibilities across teams or environments.
15. **Explain internal desirable design characteristics of software design process.**
    Internal desirable characteristics include **modularity** (dividing into manageable units), **cohesion** (each unit focuses on a single purpose), **loose coupling** (minimal inter-unit dependency), **extensibility** (easy to add features), and **reusability** (components can be reused), ensuring the design is maintainable, scalable, and adaptable over time.
16. **Write types of dispensable and shortly explain each.**
    Lazy Class: A class with minimal functionality that could be merged or removed. **Dead Code**: Unused code that serves no purpose and should be eliminated. **Speculative Generality**: Over-engineered code for hypothetical future use, adding unnecessary complexity. **Comments**: Excessive or outdated comments that clutter rather than clarify code.
17. **Which types of guidelines should be followed at the time of designing request processing strategy for a web application? Explain.**
    Use asynchronous processing to handle high loads efficiently, implement caching to reduce latency, ensure robust error handling for reliability, and design load balancing to distribute requests across servers. These guidelines enhance performance, scalability, and user experience while maintaining system stability.
18. **Enlist at least four uses of Software Architecture Patterns?**
    Software architecture patterns simplify design decisions by providing proven solutions, improve system scalability through structured growth, enhance maintainability by organizing code logically, and ensure consistency in addressing recurring problems across projects.
19. **Dispensable is a category of bad smell in code, identify the different types of dispensables in code from the given options.**
    I. **Comments**: Excessive or irrelevant comments that obscure code.
    IV. **Duplicate code**: Redundant code that should be consolidated.
    V. **Lazy class**: Underutilized class that can be removed or merged.
20. **4+1 View Model of Architecture is given below which represents the four different views, you are required to identify the viewers of each type of view and mention their names.**
    - **Logical View**: Developers (focus: system structure).
    - **Development View**: Programmers (focus: codebase organization).
    - **Process View**: System integrators (focus: runtime behavior).
    - **Physical View**: System engineers (focus: deployment).
    - **Scenarios**: Stakeholders (focus: system use cases).

Usama Farooq